SRM Institute of Science and Technology, Tiruchirapalli

Course :- Data Structures and Algorithms

Assessment :- Task 2

Topic :- Stack Operations Using Array

Title :- Code Trace Challenge - Think, Trace, Predict and Verify

Name :- K. Bhavani

Register Number :- RA2511026050305

Department :- AIML (SCSE)

Semester :- III

Scanned with OKEN Scanner

Task 1 - Code Understanding

1. Data Structure Used - Stack (Array Implementation)

The program uses a stack, which follows the LIFO (Last In First Out) principle.

2. Variables Used

Variable	Type	Purpose
stack [SIZE]	int array	Stores stack elements
top	int	Index of top element
item	int	Stores popped value
i	int	Loop variable
SIZE	Macro	Maximum stack size(5)

3. Initial Values

Variable	Initial Value
SIZE	5
top	-1
stack	Empty

4. Functions

Push (item)

→ Adds an element to the stack.

Scanned with OKEN Scanner

4. Pop()

→ Removes the top element and returns it.

display()

→ Displays all elements present in the stack.

main()

→ Calls all stack operations.

5. Conditions for

a) Overflow

Code

```
if (top == SIZE-1)
```

→ When true, Stack Overflow occurs.

b) Underflow

```
if (top == 1)
```

→ When true, Stack Underflow occurs.

6. Loop

```
for (i=0; i<=top; i++)
```

Purpose:-

→ Displays every stack element from bottom to top.

Scanned with OKEN Scanner

Task 2 – Manual Trace Table

Step	Statement /Operation	Variable Values	Stack Status	Output
1.	Initial State	top = -1	Empty	–
2.	push(10)	top = 0	10	Pushed : 10
3.	push(20)	top = 1	10 20	Pushed : 20
4.	push(30)	top = 2	10 20 30	Pushed : 30
5.	display()	top = 2	10 20 30	Stack : 10 20 30
6.	pop()	item = 30 top = 1	10 20	returns 30
7.	printf()	item = 30	10 20	Popped : 30
8.	push(40)	top = 2	10 20 40	Pushed : 40
9.	push(50)	top = 3	10 20 40 50	Pushed : 50
10.	push(60)	top = 4	10 20 40 50 60	Pushed : 60
11.	push(70)	top = 4	10 20 40 50 60	Stack Overflow
12.	display()	top = 4	10 20 40 50 60	Stack : 10 20 40 50 60

Scanned with OKEN Scanner

13.	pop()	item = 60 top = 3	10 20 40 50	Popped: 60
14.	pop()	item = 50 top = 2	10 20 40	Popped: 50
15.	pop()	item = 40 top = 1	10 20	Popped: 40
16.	pop()	item = 20 top = 0	10	Popped: 20
17.	pop()	item = 10 top = -1	Empty	Popped: 10
18.	display()	top = -1	Empty	Stack is Empty
19.	pop()	top = -1	Empty	Stack Underflow
20.	printf()	item = -1	Empty	Popped: = 1

Scanned with OKEN Scanner

Task 3 - Value Tracing

Variable Tracking

Step	top	item	Stack []
Initial	-1	-	Empty
push 10	0	10	10
push 20	1	20	10 20
push 30	2	30	10 20 30
pop	~~1~~	30	10 20
push 40	~~2~~	40	10 20 40
push 50	3	50	10 20 40 50
push 60	~~4~~	60	10 20 40 50 60
Overflow	4	70	No Change
pop	~~3~~	60	10 20 40 50
pop	2	50	10 20 40
pop	1	40	10 20
pop	0	20	10
pop	-1	10	Empty

Scanned with OKEN Scanner

Loop Variable

First display()

i	Printed
0	10
1	20
2	30

Second display()

i	Printed
0	10
1	20
2	40
3	50
4	60

Third display()

No loop executed because top = -1.

Task 4 - Output Prediction

Predicted Output

Pushed : 10

Pushed : 20

Pushed : 30

Stack : 10 20 30

Popped : 30

Scanned with OKEN Scanner

Pushed : 40

Pushed : 50

Pushed : 60

Stack Overflow

Stack : 10 20 40 50 60

Popped : 60

Popped : 50

Popped : 40

Popped : 20

Popped : 10

Stack is Empty

Stack Underflow

Popped : -1

(i) Final Stack

Empty

(ii) Final top

top = -1

(iii) Overflow

Occurs during

push (70)

Reason :- Stack is already full.

Scanned with OKEN Scanner

(iv) Underflow

Occurs during the last.

pop()

Reason:- Stack is empty.

Task 5 - What-if Analysis

→ Example:-

Original

push(60)

push(70)

→ Result:- Overflow occurs.

Modified

→ Remove

push(60)

→ Now

push(70)

is inserted successfully.

→ Reason:-

One free location is available.

Scanned with OKEN Scanner

Task 6 – Error Analysis

No logical error is present

Limitations :=

→ Fixed stack size.

→ Assay cannot grow dynamically.

→ Underflow returns -1, which may also be a valid stack element.

Suggested Improvement

→ Implement stack using a linked list or dynamic memory allocation.

Task 7 – Verification

After running the program

Predicted Output	Actual Output
Stack Overflow	Stack Overflow
Stack Underflow	Stack Underflow
Final top = -1	Final top = -1

→ **Result :=**

Prediction matches the actual output.

Scanned with OKEN Scanner

Additional Test Cases

1) Test Case 1 :=

Operations

push (5)
push (10)
display ()

Expected

Stack : 5 10

2) Test Case 2 :=

Operations

push (100)
pop ()

Expected

Popped : 100

3) Test Case 3 :=

Operations

pop ()

Expected

Stack Underflow
Popped : -1

Scanned with OKEN Scanner

Conclusion :=

> The Stack program was manually traced before execution. The predicted outputs matched the actual outputs after execution. The program correctly performed push, pop, and display operatio while handling both overflow and underflow conditions. This exercise improved the understanding of stack operations, variable tracing, function execution, and manual code analysis.

Scanned with OKEN Scanner